Attempting a SWAP-less implementation of a super-Grover quantum algorithm for optimization problems

ParityQC technical interview

Francesco Ghisoni

Abstract

These notes summarize my attempt to connect two works studied as part of a technical interview: the SWAP-less compilation framework of Klaver et al. [1] and the fault-tolerant optimization algorithm of Dalzell et al. [2]. The goal was to understand whether temporal parity compilation techniques developed for limited-connectivity hardware could be used to compile the main subroutines of the super-Grover optimization algorithm on a Linear Nearest Neighbour (LNN) architecture. I first review the SWAP-less compilation method and reproduce its main constructions, including the QFT and QAOA compilation routines. I then summarize the algorithm of Dalzell et al. [2], with emphasis on the required subroutines used in the algorithm. Finally, I analyze which of these primitives can be naturally compiled using the SWAP-less framework. The main concrete outcome is a SWAP-less LNN implementation of a multi-controlled NOT gate C^nX using $20n$ CNOT gates, CNOT depth $20n$, $18n$ single-qubit rotations, single-qubit depth $6n$ and $n+1$ ancilla qubits. I also identify arithmetic subroutines, such as the inequality test and the sin and arcsin blocks, that can be reduced to repeated applications of an adder; since these additions can be implemented using QFT-based arithmetic, they are well suited to temporal-parity compilation. Overall, the analysis suggests that while some isolated primitives can be compiled effectively, the full QSVT and block-encoding structure of the optimization algorithm is not naturally matched to temporal parity computing. A more promising direction may be to apply the two-jump idea of Dalzell et al. [2] to a conventional Trotterized adiabatic evolution for Ising-type Hamiltonians, where the required operations align more directly with the strengths of the SWAP-less compilation framework. All developed code can be found at the following GitHub page [3].

Contents

1 Introduction **3**

2 SWAP-less implementation of quantum algorithms **4**
2.1 Parity label tracking 4
2.2 Quantum circuit inspired by the LHZ code 5
2.2.1 Code development to implement temporal parity computing 6
2.3 Quantum Fourier transform 6
2.4 The Quantum Approximate Optimization Algorithm 8

3 Mind the gap: Achieving a super-Grover quantum speedup by jumping to the end **10**
3.1 Problem statement and adiabatic optimization 10
3.2 Algorithm intuition: jumping to the solution 10
3.3 Main structure of the algorithm 12
3.4 Implementation of jump step 13

4 Attempting a SWAP-less implementation of the super-Grover quantum algorithm **17**
4.1 Summary of required operations 17
4.2 Reflection operator for H_b^η 17
4.2.1 Multi-controlled NOT gates 18

4.2.2 Controlled-controlled arcsin and sin . . . 19
4.2.3 Controlled-controlled cascade of R_y rotations . . . 21
4.3 QSVT phase operator Π_ϕ . . . 21
4.3.1 One-to-many CNOT layer . . . 21
4.4 Reflection operator for H_{init} and H_{final} . . . 22
4.4.1 Controlled inequality tests and its inverse . . . 22

5 Conclusions **23**

A Brief overview of the quantum singular value transformation **24**
A.1 Block encodings . . . 24
A.2 Singular value transformations . . . 24
A.3 Quantum singular value transformation . . . 24

B Construction of reflection operators **26**

C Exact reflections for classically diagonal Hamiltonians **27**

D Derivation of fundamental primitives **27**

1 Introduction

For the technical interview I was supplied with two papers; the first, Klaver et al. [1], is a method to compile quantum algorithms on quantum hardware with limited connectivity without the need for SWAP gates, and the second, Dalzell et al. [2], is a fault-tolerant quantum algorithm for optimization problems. The task was to familiarize myself with the papers. Given the natural fit between the two works, I decided that the best way to do this was to attempt to compile the optimization algorithm of Dalzell et al. [2] assuming Linear Nearest Neighbour (LNN) connectivity, using the SWAP-less compilation techniques of Klaver et al. [1].

In these notes I show the steps that I took to achieve this task. The notes are structured as follows: in Sec. 2, I first summarize the paper of Klaver et al. [1], where I also present some methods I developed that allowed me to re-derive the results in the paper. In Sec. 3, I review the quantum algorithm of Dalzell et al. [2], with particular focus on the algorithmic steps and implementation subroutines. Finally, in Sec. 4, I attempt to compile the subroutines of Dalzell et al. [2] using the methods of Klaver et al. [1], assuming LNN connectivity. In particular, I was able to derive a SWAP-less implementation of a multi-controlled NOT gate C^nX on LNN connectivity using $20n$ CNOT gates, CNOT depth $20n$, $18n$ single-qubit rotations, a single-qubit depth $14n$ and $n+1$ ancillas for a single C^nX operation. I also identify useful arithmetic subroutines, such as the inequality test, which employ the quantum Fourier transform and can be implemented without SWAP gates using the compilation framework. All developed code can be found at the following Github page [3].

Several observations emerged from this analysis. First, an extension of the extended LHZ code to general n-qubit parities, together with systematic parity-label tracking beyond only one and two-body terms, would be useful for implementing operations such as $C^{n+1}X$. Second, the compilation of QSVT-based, block-encoding algorithms using temporal parity computing appears to be somewhat unnatural. The purpose of block encoding is to enlarge the working register so that one can represent operations such as sums of matrices, rather than only products, and implement matrix functions as polynomial transformations built from sums of polynomial terms. This is quite different from the setting in which temporal parity computing is most naturally effective, namely, as emphasized by Klaver et al. [1], the implementation of algorithms through the exponentialization of Hamiltonians, in particular Ising like Hamiltonians. Given these considerations, a more natural future direction may be to apply the two-jump idea of Dalzell et al. [2] directly to a conventional adiabatic evolution implemented through Trotterization. This direction is particularly natural for Ising spin-glass optimization, which Dalzell et al. [2] identify as one of the problem families where their strategy can be proved to work well due to favorable spectral properties, and where the resulting algorithm appears to offer an advantage over the best-known classical approaches. In fact, the QAOA compilation analysis already provides the main subroutines required to implement such an adiabatic protocol for an Ising model. In that setting, the bounds derived in Dalzell et al. [2] could help determine the appropriate size of the jumps, while the resulting time-evolution circuits would be a much better match for SWAP-less compilation through temporal parity computing.

2 SWAP-less implementation of quantum algorithms

This work introduces a compilation approach for implementing quantum algorithms on hardware with restricted connectivity without relying on SWAP gates. The central idea is to track how logical parity information flows through a circuit under entangling gates, so that non-local logical operations can be made local by changing the parity labels associated with the physical qubits. This parity-label tracking formalism is then connected to the extended Lechner-Hauke-Zoller (LHZ) architecture, which provides a geometric way of organizing the evolution of parity labels through spanning lines and local code deformations. Using this framework, the authors construct explicit SWAP-less implementations of two important algorithmic primitives, namely the quantum Fourier transform (QFT) and the Quantum Approximate Optimization Algorithm (QAOA), achieving improved resource requirements on LNN architectures compared with state of the art compilation strategies.

2.1 Parity label tracking

In this section, I introduce the notion of *parity label tracking* and explain how analyzing quantum operators through this lens provides a systematic way to decompose logical operations into physical operations. This will form the basis for the efficient implementations of the QFT and QAOA algorithms discussed later on.

Before delving into the algorithm, I will first give a brief description of the notion of parity in an n-qubit system. Parity refers to information associated with the relative configuration of a subset of qubits along a specific axis. A general product $\prod_{j\in S} Z_j$ represents the Z-parity of the subset $S \subseteq \{0,\dots,n-1\}$. In this sense, a single-qubit operator Z_i can be viewed as the trivial parity associated with the singleton set $\{i\}$, while operators such as Z_iZ_j or $Z_iZ_jZ_k$ encode two- or three-body parity information. The work of Klaver et al. [1] focuses on Z parities, where physical qubits holding multiqubit Z information are called parity qubits, while physical qubits trivially encoding single-qubit Z information are referred to as *base qubits*.

The parity-label tracking formalism makes use of this notion explicitly by assigning to each physical qubit a label indicating which logical Z-parity is currently represented by a local physical Z operator. At the initial time τ_0, each physical qubit a is assigned the parity label $P_a^{(0)} = \{a\}$, so that each physical qubit initially represents the corresponding single-qubit logical operator. The labels are then updated whenever $\mathrm{CNOT}_{c\to t}$ gates are applied, where c and t denote the control and target qubits, respectively. The application of a set of such gates defines a discrete time step, since it changes the parity labels associated with the physical qubits. More precisely, if the system is at time step $\lambda-1$ and the current labels are $P_a^{(\lambda-1)}$, then the application of $\mathrm{CNOT}_{c\to t}$ updates the labels according to

$$P_c^{(\lambda)} = P_c^{(\lambda-1)}, \tag{1}$$

and

$$P_t^{(\lambda)} = P_c^{(\lambda-1)} \Delta P_t^{(\lambda-1)}, \tag{2}$$

where Δ denotes the symmetric difference between sets. Sets of CNOT gates applied during time step λ will be denoted by a unitary operator C_λ.

Using this notation, it is possible to see how local unitary operations on physical qubits can effectively implement multiqubit unitaries on the logical system. For example, a physical operator Z_a acting on a qubit with label $P_a^{(\lambda)}$ can be understood as the logical operator $Z_a = \prod_{j\in P_a^{(\lambda)}} \bar{Z}_j$, where the bar denotes the logical nature of the operation. Similarly, the parity labels can also be used to determine how to apply logical X operators. Since the labels track information in the Z basis, applying a logical single-qubit operator $\bar{X}_j$ generally requires applying physical X operators to all physical qubits whose labels contain the logical index j.

These notions extend naturally to arbitrary rotations using the identity

$$C^\dagger e^{i\theta M} C = e^{i\theta C^\dagger M C}, \tag{3}$$

which holds for any unitary C and operator M. In this way, physical single-qubit rotations can be interpreted as logical parity rotations in the frame defined by the current parity labels. This allows one

to express logical unitary operators $\bar{U}$ through a sequence of physical X and Z rotations interleaved with CNOT layers. In particular, one may write

$$\bar{U} = \prod_{\lambda} \prod_{j} \bar{U}^{X}_{\lambda,j}(\theta_j) \bar{U}^{Z}_{\lambda,j}(\phi_j), \tag{4}$$

with $\bar{U}^{X}_{\lambda,j}(\theta_j) = \exp\left(i\theta_j \prod_{k \in Q_j^{(\lambda)}} \bar{X}_k\right)$, and $\bar{U}^{Z}_{\lambda,j}(\phi_j) = \exp\left(i\phi_j \prod_{k \in P_j^{(\lambda)}} \bar{Z}_k\right)$. Here, $Q_j^{(\lambda)}$ denotes the analogous label set obtained when tracking the transformation of operators in the X basis. This unitary decomposition is possible because physical X and Z rotations together with CNOT gates form a universal gate set.

2.2 Quantum circuit inspired by the LHZ code

The starting point of the construction is the Lechner-Hauke-Zoller (LHZ) architecture, which was originally introduced as a way to represent all-to-all connected Ising models using only local interactions between physical qubits [4]. The central idea is to replace n logical two-body interactions by $\frac{n(n-1)}{2}$ physical qubits and $\frac{(n-1)(n-2)}{2}$ constraints.

A spatial extension of this architecture was introduced in Ref. [5]. The construction can be understood as placing two LHZ trees on top of one another, with one of the trees flipped upside down as well as n base qubits used as a layer between the connection. This construction has periodic boundary conditions on the horizontal axis. The important feature of this extended layout is that any connected line of physical qubits running from the top boundary to the bottom boundary contains a complete set of logical information. Such a line is referred to as a *spanning line*.

The local consistency constraints of the extended LHZ layout are encoded in triangular boundary stabilizers and bulk plaquette stabilizers. In terms of parity labels, the defining condition is that the symmetric difference of all labels around a valid stabilizer is empty. For a plaquette with labels P_1, P_2, P_3, P_4 and a triangle with labels T_1, T_2, T_3, this can be written as

$$P_1 \Delta P_2 \Delta P_3 \Delta P_4 = \emptyset, \tag{5}$$

and

$$T_1 \Delta T_2 \Delta T_3 = \emptyset. \tag{6}$$

Within half of the horizontal period of the extended architecture, the spanning lines collectively contain all single-qubit and two-qubit Z parity labels. This is the property that makes the construction useful as a guide for circuit compilation.

The key idea of temporal parity computing is to reinterpret the horizontal direction of the extended LHZ layout as time. Instead of physically realizing the full two-dimensional architecture, one only stores a single spanning line at a time on a one-dimensional chain of physical qubits. At any given time, only the logical operations whose labels appear on the current spanning line can be applied locally. The quantum circuit then evolves this chain from one spanning line to the next. This evolution is guided by the local stabilizer structure of the extended LHZ code. If the current spanning line contains three labels of a bulk plaquette, it can be deformed toward the missing plaquette node by updating the appropriate physical qubit through CNOT gates controlled by the two neighbouring plaquette nodes, consistently with the plaquette condition in Eq. 5. Similarly, for an upper or lower boundary triangle, the spanning line can be deformed toward the missing triangle node by applying a single CNOT whose control and target are chosen according to the local geometry of the triangle, as encoded by Eq. (6). In the spatial picture this corresponds to moving the spanning line across a stabilizer, while in the circuit picture it corresponds to updating the parity labels carried by the CNOTs between nearest neighbour physical qubits.

The extended LHZ architecture therefore provides a geometric prescription for generating the CNOT circuits needed to change parity frames, while the temporal interpretation turns this prescription into a space-efficient compilation method on a LNN architecture.

2.2.1 Code development to implement temporal parity computing

In the following I explain the code development that I carried out to implement the SWAP-less compilation technique. These codes are the foundations that allow me to successfully repeat the compilation of the QFT and QAOA. All code can be found at the following GitHub link [3] in the `Scripts` folder.

To make the implementation of temporal parity computing modular, I divided the code into three main components. The first module constructs the extended LHZ architecture, the second module handles generic operations on spanning lines, and the third module contains generic compiler utilities. Algorithm-specific routines are then built on top of these modules.

The first module, `Scripts/LHZ_code.py`, is responsible for constructing the extended LHZ layout. The central function is `build_extended_lhz_array`, which returns a two-dimensional array whose entries are either `None` or parity labels. The construction follows the horizontal update rules of the extended LHZ architecture, so that each valid node of the array is assigned either a singleton label (i) or a two-body parity label (i, j). The module also contains the helper function `symdiff_labels`, which computes the symmetric difference of a collection of parity labels.

The second module, `Scripts/Spanning_lines.py`, contains the generic tools needed to work with spanning lines in the extended LHZ layout. In this module, coordinates are always treated with the horizontal periodic boundary conditions of the architecture. The code can translate between coordinate representations of spanning lines and their corresponding parity-label representations, which is useful both for checking results and for deciding where logical operators are available. It also builds the local triangular and plaquette stabilizer structures of the architecture by selecting the local faces whose parity labels satisfy the empty symmetric-difference condition of Eq. (5) and Eq. (6). The code then uses them to identify neighbouring spanning lines, in particular right-moving face crossings, and to translate each geometric move into the corresponding sequence of CNOT operations. In this way, the module provides the basic bridge between the geometry of the extended LHZ code and the circuit operations required for temporal parity computing.

The third module, `Scripts/Compiler_code.py`, contains the generic tools needed to translate logical operations into physical operations on a chosen spanning line. Given a current parity frame, the code checks whether a logical R_Z, R_X, or Hadamard operation is locally available and identifies the physical qubit on which it should be applied. For R_Z rotations, this means finding a physical qubit whose parity label matches the required logical Z support, while for R_X rotations the code checks whether the operation is local, i.e. if the logical index appears in exactly one parity label of the current spanning line. The compiler module also implements the commutation rules used during scheduling, allowing operations to be moved earlier only when this does not change the implemented unitary. Finally, it provides simple resource-estimation utilities, such as CNOT count, CNOT depth, single-qubit count, and single-qubit count. These routines are independent of the specific algorithm and can therefore be reused by the QFT and QAOA compilers.

2.3 Quantum Fourier transform

The first algorithm considered for a SWAP-less compilation is the QFT. In its standard circuit form, the QFT is implemented through a sequence of Hadamard gates and controlled-phase gates as follows:

$$U_{\mathrm{QFT}} = \prod_{i=1}^{n} \left[H_i \prod_{j=i+1}^{n} \mathrm{CP}_{ij} \left(\frac{\pi}{2^{j-i}} \right) \right], \tag{7}$$

where the controlled-phase gate is defined as

$$\mathrm{CP}_{jk}(\theta) = \mathrm{diag}(1, 1, 1, e^{i\theta}). \tag{8}$$

To express the QFT in a form compatible with parity-label compilation, the controlled-phase gates are first decomposed into logical Z-parity rotations using:

$$\overline{\mathrm{CP}}_{ij}(\theta) = R_{\mathrm{Z}}^{(i)} \left(\frac{\theta}{2} \right) R_{\mathrm{Z}}^{(ij)} \left(-\frac{\theta}{2} \right) R_{\mathrm{Z}}^{(j)} \left(\frac{\theta}{2} \right), \tag{9}$$

where $R_{\mathrm{Z}}^{(ij)}$ denotes a logical parity rotation generated by $\bar{Z}_i\bar{Z}_j$. Similarly, the Hadamard gates can be decomposed as

$$\bar{H}_i = \bar{R}_{\mathrm{Z}}^{(i)}\left(\frac{\pi}{2}\right)\bar{R}_{\mathrm{X}}^{(i)}\left(\frac{\pi}{2}\right)\bar{R}_{\mathrm{Z}}^{(i)}\left(\frac{\pi}{2}\right). \tag{10}$$

This gives a representation of the QFT in terms of logical R_Z and R_X rotations, which can then be localized using the parity labels appearing on the spanning lines.

In the code implementation, found at `Scripts/QFT_gates.py`, I first generate the standard QFT gate list in terms of Hadamard and controlled-phase gates. This list is then converted into logical R_Z and R_X rotations using Eqs. (9) and (10). At this stage, several simplifications can already be made. Since all Z-type parity rotations commute with one another, the code moves commuting R_Z rotations when allowed and combines rotations of the same type by adding their angles, which reduces the number of single-qubit logical R_Z rotations. The first R_Z on the first logical qubit is moved at the end to maintain an optimal circuit depth. The first and last Hadamard gates are kept explicitly rather than decomposed, since they are naturally local on the initial and final spanning lines, respectively.

The spanning lines used for the QFT follow the left-to-right temporal parity construction. The initial spanning line is the data-qubit line,

$$[(0,), (1,), \ldots, (n-1,)], \tag{11}$$

and the final spanning line is the reversed data-qubit line,

$$[(n-1,), (n-2,), \ldots, (0,)]. \tag{12}$$

The code, found at `Scripts/QFT_spanning_lines.py`, generates the intermediate lines by repeatedly identifying the allowed right-moving local deformations of the current spanning line. These deformations are obtained from the triangular and plaquette stabilizer faces of the extended LHZ layout. When a row reaches its final reversed-output label, it is frozen so that the procedure terminates at the correct output line.

The compiler, with the code found at `Scripts/QFT_compiler.py`, then walks through the QFT operation list and the precomputed spanning-line schedule simultaneously. Whenever the logical operation is local in the current parity frame, it is converted into a physical operation on the corresponding qubit. If no operation is currently local, without breaking commutation rules, the compiler advances to the next spanning line and inserts the CNOT gates associated with that geometric move. The scheduling also uses commutation relations: R_X rotations and two-body R_Z rotations may be moved earlier when they commute with all preceding unexecuted operations, while single-body R_Z rotations are locked to maintain the optimal single qubit depth. This produces a SWAP-less implementation of the QFT on a LNN architecture. A $n = 5$ qubit QFT is plotted in Fig. 1 using the `Scripts/Circuit_plotter.py` code, which takes as input the list of compiled operations and uses Pennylane [6] to plot the circuit directly from those. Furthermore, numerical tests, which can be found in the file `Notebooks/QFT.ipynb`, show that the scaling for CNOT count, CNOT depth, single qubit count and single qubit depth are respected.

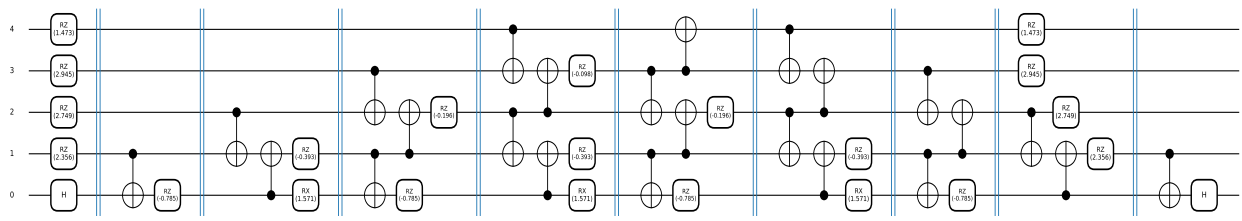


Figure 1: Quantum circuit for the 5-qubit QFT after compilation with parity temporal codes. The circuit consists of single-qubit rotations H, R_x, R_z, CNOT gates. Angles are given in radians. Code related to the development of this circuit can be found at the following Github [3] in the `Notebooks/QFT.ipynb` notebook.

2.4 The Quantum Approximate Optimization Algorithm

Optimization problems are an important class of problems with many real-world applications. In the setting considered here the goal is to find a bit string $z^* \in \{+1, -1\}^n$ that minimizes an efficiently computable classical cost function $H : \{+1, -1\}^n \to \mathbb{R}$. In general, the runtime for algorithms for these problems is expected to be exponential in the number of variables.

The Quantum Approximate Optimization Algorithm (QAOA) [7] is a hybrid quantum-classical algorithm designed to approximately solve such optimization problems by preparing a parametrized quantum state whose measurement outcomes are biased towards low-cost bit strings.

The first step is to promote the classical objective function to a quantum Hamiltonian by mapping each binary variable z_i to a spin operator $\bar{Z}_i$. For many relevant problems this gives an Ising-type problem Hamiltonian of the form:

$$H_P = \sum_{k=1}^{n} \sum_{j<k} J_{jk} \bar{Z}_j \bar{Z}_k + \sum_{j=1}^{n} h_j \bar{Z}_j, \tag{13}$$

where J_{jk} are the two-body coupling coefficients and h_j are local fields. The role of H_P is to encode the cost function into the energy landscape of the quantum system, so that low-energy computational basis states correspond to good candidate solutions of the original optimization problem.

The second ingredient is a mixing Hamiltonian, which is usually chosen as

$$H_X = \sum_{j=1}^{n} \bar{X}_j. \tag{14}$$

The corresponding unitary are then:

$$U_P(\gamma) = \exp(i\gamma H_P), \qquad U_X(\beta) = \exp(i\beta H_X). \tag{15}$$

Starting from the uniform superposition state $|+\rangle^{\otimes n}$, these unitaries are applied in an alternating sequence for p layers, producing the variational state

$$|\psi(\boldsymbol{\beta}, \boldsymbol{\gamma})\rangle = \prod_{\ell=1}^{p} U_X(\beta_\ell) U_P(\gamma_\ell) \, |+\rangle^{\otimes n} \, . \tag{16}$$

The parameters $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$ are then optimized by minimizing the expectation value of the problem Hamiltonian,

$$E(\boldsymbol{\beta}, \boldsymbol{\gamma}) = \langle \psi(\boldsymbol{\beta}, \boldsymbol{\gamma}) | \, H_P \, | \psi(\boldsymbol{\beta}, \boldsymbol{\gamma}) \rangle \, . \tag{17}$$

After optimization, measurements of the final state are expected to return bit strings with low cost.

For the purposes of the present work, I follow the approach of Ref. [1] and focus on compiling the two elementary QAOA unitaries $U_P(\gamma)$ and $U_X(\beta)$ separately. The gate generation, found at `Scripts/QAOA_gates.py`, is therefore simpler than for the QFT. The problem unitary $U_P(\gamma)$ is generated directly as a list of logical R_Z rotations: two-body terms $J_{jk}\bar{Z}_j\bar{Z}_k$ are mapped to parity rotations on labels (j, k), while one-body terms $h_j\bar{Z}_j$ are mapped to rotations on singleton labels (j). The mixer unitary $U_X(\beta)$ is generated as a list of logical R_X rotations. Since all terms within H_P commute with each other, and all terms within H_X commute with each other, no additional decomposition or compression step is required at this stage.

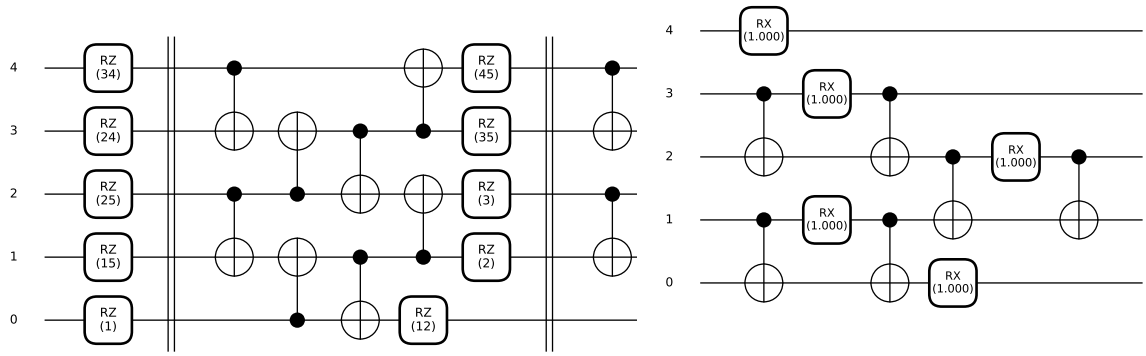
The spanning-line construction, found at `Scripts/QAOA_spanning_lines.py`, for $U_P(\gamma)$ differs from the QFT case. As indicated in Ref. [1], the first spanning line for the implementation of U_P is chosen as a zig-zag line through the extended LHZ architecture. In the code implementation this is achieved by starting at the $(0,)$ data qubit on coordinate $[0, 0]$ of the extended LHZ architecture, and then alternating between south-west and south-east moves, with periodic boundary conditions in the horizontal direction, until reaching the final row of the architecture. For example, for $n = 5$ this gives an initial spanning line

$$[(0,), (0, 4), (1, 4), (1, 3), (2, 3)], \tag{18}$$

which is the reversed line as the one presented in figure 4a of Ref. [1]. Subsequent spanning lines are generated by moving the line through the stabilizer faces of the extended LHZ layout. Including the initial line, the schedule contains a fixed number of spanning lines, given by $\left\lceil \frac{n-1}{2} \right\rceil + 1$. The generation procedure is then stopped once this number of lines has been produced. For odd n, each transition between consecutive spanning lines is obtained by applying two global right-moving updates across the available stabilizers. For even n, the same rule is used except for the final transition, which consists of only a single right-moving update. This distinction ensures that the schedule terminates correctly while preserving the CNOT number and depth scaling of $n^2 - 2n + 1$ and $2n - 2$ respectively for both even and odd n.

The spanning-line input for $U_X(\beta)$ is instead chosen to be the final spanning line of the $U_P(\gamma)$ schedule, following the construction of Ref. [1]. This choice is also natural from a circuit perspective, since a QAOA layer applies $U_X(\beta)$ immediately after $U_P(\gamma)$, allowing the mixer unitary to be implemented directly from the parity frame reached at the end of the problem unitary.

Finally I move to the compiler, found at `Scripts/QAOA_compiler.py`, for both $U_P(\gamma)$ and $U_X(\beta)$. The compiler for $U_P(\gamma)$ is direct. Since all terms in the problem unitary are Z-type rotations and mutually commute, the compiler scans the available spanning lines and applies each logical R_Z rotation as soon as its required label appears on the current line. Between consecutive spanning lines, the corresponding CNOT sequence generated by the stabilizer-face crossings is inserted. The compiler for $U_X(\beta)$ uses a different structure, because logical X rotations are implemented along X logical lines rather than directly through the Z-parity labels stored in the extended LHZ architecture. These logical X lines span at most two neighboring physical qubits, and at least one logical X operator is localized on a boundary. As a result, the mixer unitary can be implemented with constant two-qubit gate depth and at most $2n - 2$ CNOT gates, followed by local R_X rotations on the appropriate physical qubits. The plot of the compiled unitaries for $n = 5$ can be seen in Fig. 2, where Fig. 2a illustrates a single layer of $U_P(\gamma)$ whilst Fig. 2b illustrates a single layer of $U_X(\beta)$. Numerical tests in `Notebooks/QAOA.ipynb` confirm that the compiled circuits reproduce the expected scaling of the CNOT count, CNOT depth, single-qubit count, and single-qubit depth for both unitaries.



(a) Quantum circuit for the 5-qubit $U_P(\gamma)$ unitary. Angles represent the logical labels of the R_Z rotations.

(b) Quantum circuit for the 5-qubit $U_X(\beta)$ unitary.

Figure 2: Quantum circuit for the 5-qubit QAOA unitaries $U_P(\gamma)$ and $U_X(\beta)$ after compilation with parity temporal codes. Code related to the development of this gate can be found at the following Github [3] in the `Notebooks/QAOA.ipynb` notebook.

3 Mind the gap: Achieving a super-Grover quantum speedup by jumping to the end

In this section, I introduce the optimization algorithm of Dalzell et al. [2], which provides a framework for obtaining a super-Grover speedup for certain families of exact binary optimization problems. We first describe the scope of the algorithm and its connection with adiabatic quantum optimization, where the objective is to move from the easily prepared ground state of an initial Hamiltonian to the ground space of a classical cost Hamiltonian. However, rather than following a slow continuous adiabatic path, the algorithm can be understood as a two-step procedure: it first prepares an intermediate ground state through a restricted jump, thereby amplifying its overlap with the optimal eigenspace, and then performs a second jump to the optimal eigenspace, which encodes the solution. Although the algorithm can be motivated from the perspective of adiabatic optimization, the circuit implementation considered below is expressed in terms of modern block-encoding primitives related to Grover's algorithm.

3.1 Problem statement and adiabatic optimization

As with the QAOA (Sec. 2.4), this work remains in the paradigm of optimization problems. To recap, in optimization problems the goal is to find a bit string $z^* \in \{+1, -1\}^n$ that minimizes an efficiently computable classical cost function $H : \{+1, -1\}^n \to \mathbb{R}$. From now on in the notes, H denotes the diagonal $2^n \times 2^n$ Hermitian operator for which $\langle z| H |z\rangle = H(z)$. In general, the runtime for algorithms for these problems is expected to be exponential in the number of variables.

The work views optimization through the lens of the adiabatic quantum algorithm [8], [9]. In the context of optimization, the algorithm works by first defining an initial Hamiltonian of the form:

$$H_{\text{init}} = -\frac{X}{n}, \tag{19}$$

where $X := \sum_{i=1}^{n} X_i$, and X_i is the Pauli-X operator acting on qubit i. A key property of the initial Hamiltonian is that its ground state is easy to prepare. In the case of Eq. (19), this is satisfied since the unique ground state is the uniform superposition state $|+\rangle^{\otimes n}$. Then define the final Hamiltonian:

$$H_{\text{final}} = \frac{H}{|E^*|}, \tag{20}$$

where $E^* < 0$ is the minimum energy, so that $|E^*|$ denotes its absolute value. The ground space of this Hamiltonian is spanned by computational basis states corresponding to optimal solutions. If the solution is unique, its ground state is simply $|z^*\rangle$. Finally, the interpolation Hamiltonian is defined as:

$$H_b = H_{\text{init}} + b\, H_{\text{final}}, \tag{21}$$

which at $b = 0$ has ground state $|+\rangle^{\otimes n}$, while in the limit $b \to \infty$, the ground space approaches the ground space of $H/|E^*|$ and therefore contains the optimal solution.

Adiabatic quantum optimization aims at finding an optimal solution z^* by preparing a state in the ground space of H_{final} by evolving $|+\rangle^{\otimes n}$, the ground state of H_{init}, through H_b. This is guaranteed to work for sufficiently slow evolution as long as the adiabatic theorem conditions hold [10]. This requires the time evolution operator of H_b to have no energy level crossings. In the case of no level crossings but an exponentially small energy gap, the runtime can become exponentially large, eliminating the advantage one hopes to obtain. In addition, when performing the algorithm on a digital quantum device, any practical simulation of the continuous evolution introduces time-evolution errors, which must be controlled by choosing a sufficiently fine discretization or by using accurate Hamiltonian simulation primitives.

3.2 Algorithm intuition: jumping to the solution

The first building block of the algorithm, originally developed in the short-path algorithm of Hastings [11], can be intuitively understood through the lens of an adiabatic optimization. In particular,

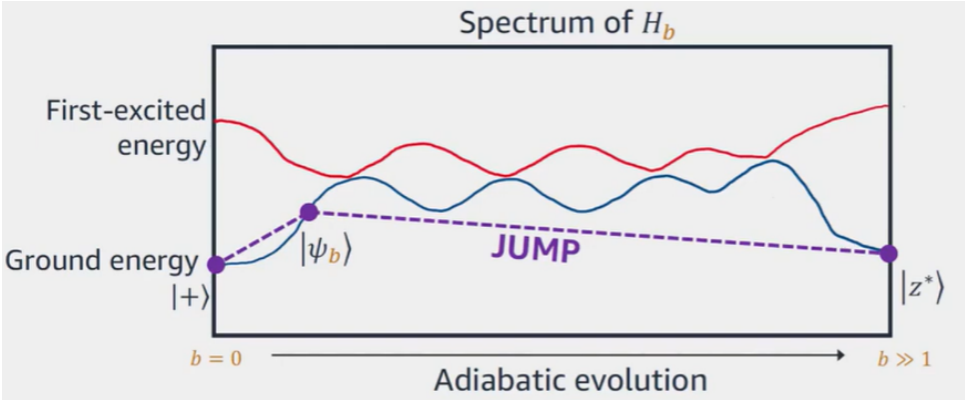


Figure 3: A schematic representation of the algorithm by Dalzell et al. [2]. From this it is clear that the algorithm consists of two steps: (i) a restricted jump from the ground state $|+\rangle^n$ to an intermediate state $|\psi_b\rangle$, which is the ground state of Eq. (25), and (ii) a large jump from $|\psi_b\rangle$ to the solution $|z^*\rangle$ which aims at avoiding all the regions with a small energy gap. This image was taken from the 2023 QIP talk of Dalzell which can be found here.

the idea is that the total error of an adiabatic-style procedure that uses a single large jump, while avoiding regions with an exponentially small spectral gap, can be smaller than the total error of a conventional adiabatic evolution that takes many small steps through regions where the spectral gap becomes exponentially small. In this sense, the algorithm replaces a slow continuous interpolation by two transitions between carefully chosen ground spaces. A schematic of the algorithm can be seen in Fig. 3. It is worth noting that the algorithm itself is not an adiabatic evolution, rather it can be understood as a double step Grover search.

To see how these jumps are implemented, consider two Hamiltonians H_1 and H_2. Suppose that $|\psi_1\rangle$ is the ground state of H_1 and that Π_2 is the projector onto the ground space of H_2. A jump from H_1 to H_2 aims to implement a unitary U that acts as

$$U\,|\psi_1\rangle = \frac{\Pi_2|\psi_1\rangle}{\|\Pi_2|\psi_1\rangle\|}. \tag{22}$$

The success probability of directly projecting $|\psi_1\rangle$ onto the ground space of H_2 is determined by the squared overlap

$$p_{\text{succ}} = \|\Pi_2|\psi_1\rangle\|^2. \tag{23}$$

Operationally, the jump can be achieved using the quantum singular value transformation (QSVT) [12] alongside fixed-point amplitude amplification [13]. It is worth noting that an exponentially small success probability will require an exponential number of repetitions of the amplitude amplification oracle.

The second insight of the proposed algorithm is to apply a piecewise-linear transformation $g_\eta : [-1, \infty) \to [-1, 0]$ to the final Hamiltonian in Eq. (21). The function g_η takes the form:

$$g_\eta(x) = \min\left(0, \frac{x + 1 - \eta}{\eta}\right), \qquad 0 \le \eta < 1. \tag{24}$$

The aim of this function is to keep only sufficiently low-energy eigenstates relevant while mapping higher-energy eigenstates to zero. Since $E^* < 0$, g_η preserves the optimal eigenspace as the lowest-energy subspace. Therefore, the interpolation Hamiltonian used by the algorithm is:

$$H_b^\eta = -\frac{X}{n} + b\, g_\eta\left(\frac{H}{|E^*|}\right), \tag{25}$$

whose ground state is denoted by $|\psi_b\rangle$.

3.3 Main structure of the algorithm

The algorithm has a simple high-level structure: it starts from the easily prepared ground state $|+\rangle^{\otimes n}$ of H_{init} (Eq. (19)), performs a first jump to the ground state $|\psi_b\rangle$ of the modified Hamiltonian H_b^η (Eq. (25)), and then performs a second jump to the ground space of the classical cost Hamiltonian H_{final} (Eq. (20)). A final computational-basis measurement then returns an optimal eigenstate z^*. The procedure is summarized in Algorithm 1.

Algorithm 1 Main algorithm

Require: A cost function H, optimal value E^*, parameter η, and parameter b, which together define H_b^η in Eq. (25).

Ensure: An optimal eigenstate z^* for H.

1: Prepare $|+\rangle^{\otimes n}$, the ground state of $-X/n$.
2: Prepare $|\psi_b\rangle$ up to exponentially small error by performing the jump $-\frac{X}{n} \longrightarrow H_b^\eta$.
3: Prepare $\frac{\Pi^*|\psi_b\rangle}{\|\Pi^*|\psi_b\rangle\|}$ up to exponentially small error by performing the jump $H_b^\eta \longrightarrow \frac{H}{|E^*|}$, where Π^* is the projector onto the ground space of H.
4: Measure in the computational basis to produce $|z^*\rangle$.

After preparing the initial state $|+\rangle^{\otimes n}$, the algorithm first performs the jump

$$-\frac{X}{n} \longrightarrow H_b^\eta. \tag{26}$$

This prepares the intermediate ground state $|\psi_b\rangle$, up to exponentially small error. The role of $|\psi_b\rangle$ is to serve as an intermediate state that has more weight on low-energy eigenstates than the uniform superposition. The success of this step is strongly influenced by the choice of b. In particular, b controls how far the intermediate Hamiltonian moves away from the initial Hamiltonian $-X/n$ and toward the transformed cost Hamiltonian $g_\eta(H/|E^*|)$. For small values of b, the ground state $|\psi_b\rangle$ remains close to the initial uniform superposition, so the first jump has a relatively large success probability. As b is increased, $|\psi_b\rangle$ becomes more concentrated on low-energy computational-basis eigenstates, which is useful for the second jump, but one also risks approaching regions where the spectral gap becomes small. The paper provides theoretical bounds on the value of b that guarantee sufficiently favorable spectral properties, with numerical evidence suggesting that much larger values of b may work than those guaranteed by the proof.[1] Overall, the paper provides analytical bounds showing that the success probability of preparing $|\psi_b\rangle$ is sufficiently large for this step not to dominate the total runtime.

The algorithm then performs the second jump

$$H_b^\eta \longrightarrow \frac{H}{|E^*|}. \tag{27}$$

This step projects the intermediate state $|\psi_b\rangle$ onto the optimal eigenspace of the original cost Hamiltonian. Let Π^* denote the projector onto the ground space of H. The target state of this jump is then

$$\frac{\Pi^* |\psi_b\rangle}{\|\Pi^* |\psi_b\rangle\|}, \tag{28}$$

[1] The choice of b is made rigorous through a spectral-density tail bound. Let $C(E) := |\{|z\rangle : H(z) \leq E\}|$ denote the number of computational-basis eigenstates of the classical Hamiltonian H with energy at most E. The relevant tail bound is

$$C\big((1-\eta)E^*\big) \leq 2^{(1-\gamma)n}.$$

Since the total Hilbert-space dimension is 2^n, the parameter γ measures how sparse the low-energy tail of the spectrum is. Under this tail-bound assumption, the paper proves that the required spectral conditions hold whenever

$$b \leq \frac{\ln(2)\gamma}{2+\ln(2)} \approx 0.257\gamma.$$

which is prepared up to exponentially small error.

The parameter η plays an important role in this step because it determines which low-energy eigenstates are kept energetically relevant by the function g_η. More precisely, g_η maps eigenstates with normalized energy above the threshold $-(1-\eta)$ to zero, while acting linearly on eigenstates below this threshold. Thus, η controls the size of the low-energy window that contributes to the structure of $|\psi_b\rangle$. A larger value of η allows the transformed Hamiltonian to retain information about a broader set of low-energy eigenstates, which can increase the overlap of $|\psi_b\rangle$ with the optimal eigenspace. However, this must be balanced against the spectral conditions required for the first jump, since the same choice of η also enters the bound that affects the rigorous range of allowed values of b.

It is important to note that the overlap $\|\Pi^* |\psi_b\rangle\|$ may still be exponentially small, but the key point is that it can be exponentially larger than the overlap obtained from the uniform superposition. If, for some constant $c > 0$ independent of the system size n, one has

$$\|\Pi^* |\psi_b\rangle\| \gtrsim 2^{-(1/2-c)n}, \tag{29}$$

then fixed-point amplitude amplification gives a runtime scaling as

$$O^*(2^{(1/2-c)n}), \tag{30}$$

which is a super-Grover speedup [14]. Here, $O^*(\cdot)$ suppresses factors polynomial in n.

Once the projection onto the optimal eigenspace has been performed, the final step is to measure in the computational basis. Since the resulting state is supported on the ground space of the classical Hamiltonian H, this measurement returns an optimal computational-basis eigenstate $|z^*\rangle$, and therefore the corresponding optimal solution z^*.

The paper shows that this mechanism can be made rigorous for certain families of binary optimization problems. Some of the corresponding runtime guarantees, together with the best-known classical exponents, are summarized in Table 1. Qualitatively, the proof shows that the transformed Hamiltonian H_b^η can retain sufficiently favorable spectral properties, allowing the first jump to avoid an exponentially small-gap region. It also proves that the intermediate ground state $|\psi_b\rangle$ can have sufficiently large overlap with the optimal eigenspace to make the second jump faster than Grover search. Importantly, for the problem families considered, the improvement in the exponent is by a constant c that is independent of n, and therefore does not vanish as the system size grows. The technical details require spectral-density estimates, gap bounds, and overlap bounds, and are left to the original paper.

Problem	Dalzell et al. [2]	Classical runtime
k-spin model	$0.5 - \Omega(1/k^3)$	1
SK model	0.49983	0.45 [15]
3-SAT	0.49999948	0.39 [16]
k-SAT	$0.5 - \Omega(1/(2^{3k}k^3))$	$1 - \Omega(1/k)$ [16]

Table 1: Comparison between the runtime exponent achieved by the algorithm of Dalzell et al. [2] and the best-known classical runtime exponents for problem families where a rigorous guarantee is provided. The numbers correspond to the exponent a in a runtime of the form $O^*(2^{an})$, where $O^*(\cdot)$ suppresses polynomial factors in n. For Grover search [14], $a = 0.5$. For the k-spin model, the classical exponent is listed as 1 because the comparison is against exhaustive search; to the best of my knowledge, no classical algorithm with a provably better exponent is known for the general k-spin model.

3.4 Implementation of jump step

I now describe the circuit-level ingredients required to implement the two jump steps appearing in Algorithm 1. The details of this subsection require knowledge of block encoding, singular value transformations and the QSVT [12]. A brief overview of these topics is given in Appendix A.

Proposition 1 (The generic jump primitive.)**.** *Suppose that H_1 and H_2 are n-qubit Hamiltonians. For each $j \in \{1,2\}$, assume that U_j is a (κ_j, a)-block encoding of H_j, and that controlled-U_j can also be implemented. Also assume an energy thresholds E_j, spectral gaps Δ_j, a target precision δ, and a lower bound $p \leq \|\Pi_2 |\psi_1\rangle\|^2$ on the squared overlap with the target subspace. Here $|\psi_1\rangle$ denotes the unique eigenstate of H_1 with energy at most E_1, and Π_2 denotes the projector onto the eigenspace of H_2 with energy at most E_2.*

For each $j \in \{1,2\}$, further assume that at least one of the following two conditions holds:

(i) H_j has no eigenvalues in the interval $(E_j, E_j + \Delta_j)$;

(ii) H_j can be represented as a diagonal matrix in either the computational basis or the Hadamard basis, and each of its entries in that basis can be computed in classical time $\mathrm{poly}(n)$.

Under these assumptions, the jump primitive implements a unitary U satisfying

$$\left\| U\left(|\psi_1\rangle |0\rangle^{\otimes a}\right) - \frac{\Pi_2 |\psi_1\rangle}{\|\Pi_2 |\psi_1\rangle\|} |0\rangle^{\otimes a} \right\| \leq \delta. \tag{31}$$

using $\mathrm{poly}(n)$ *additional gates and, for a constant D and each $j \in \{1,2\}$, uses*

$$\frac{D\kappa_j}{\Delta_j \sqrt{p}} \log(\delta^{-1}) \log\Big(p^{-1/2}\delta^{-1} \log(\delta^{-1})\Big) \tag{32}$$

calls to U_j and controlled-U_j. If H_j is diagonal in the computational basis or the Hadamard basis and each of its entries in that basis can be computed in $\mathrm{poly}(n)$ *classical time, then no calls to U_j are necessary for that Hamiltonian, and the corresponding gap parameter Δ_j is irrelevant. When acting with U on $|\psi_1\rangle$, we say that we are performing the jump $H_1 \longrightarrow H_2$.*

Implementation. The implementation proceeds in two main stages.

Step 1: Reflection operators. Defining the step function $q_j(x)$ as:

$$q_j(x) = \begin{cases} -1, & x \leq 0, \\ +1, & x > 0. \end{cases} \tag{33}$$

The corresponding exact reflection is

$$R_j = q_j\left(\frac{H_j - E_j - \Delta_j/2}{2\kappa_j}\right). \tag{34}$$

The shift by $E_j + \Delta_j/2$ places the midpoint of the promised spectral gap at zero, while the division by $2\kappa_j$ normalizes the shifted Hamiltonian so that its spectrum lies within $[-1,1]$. This operator applies a -1 phase to the low-energy eigenspace of H_j and acts as the identity on the orthogonal subspace. Since the exact step function is discontinuous, it cannot be applied with the QSVT. Therefore, it is replaced by an polynomial approximation q'_j with precision δ'. The approximate reflection is then defined as

$$R'_j = q'_j\left(\frac{H_j - E_j - \Delta_j/2}{2\kappa_j}\right). \tag{35}$$

The polynomial q'_j is odd and bounded on $[-1,1]$, and can therefore be implemented using the QSVT [12]. More details on the construction of these reflection operators can be found in Appendix B.

More precisely, QSVT is applied to a block encoding of the shifted and rescaled Hamiltonian

$$\frac{H_j}{2\kappa_j} - \frac{E_j + \Delta_j/2}{2\kappa_j} I. \tag{36}$$

This block encoding can be constructed from a controlled version of the block encoding U_j, together with a block encoding of the identity, using a linear-combination-of-block-encodings construction. Applying QSVT with the polynomial q'_j then gives an implementation of the approximate reflection

R'_j. Since the spectrum is promised to avoid the interval $(E_j, E_j + \Delta_j)$, every eigenvalue lies in a region where q'_j approximates the step function, and therefore

$$\|R_j - R'_j\| \leq \delta'. \tag{37}$$

If condition (ii) holds for either H_1, H_2, or both, then the corresponding reflection operator can be implemented exactly without using QSVT. Suppose first that H_j is diagonal in the computational basis and that T bits are sufficient to represent its diagonal entries $(H_j)_{ii}$. Since these entries can be computed efficiently by assumption, it is possible to use reversible classical arithmetic to implement

$$|i\rangle |0\rangle^{\otimes A} \longmapsto (-1)^{(H_j)_{ii} \leq E_j} |i\rangle |0\rangle^{\otimes A}, \tag{38}$$

where $A = \text{poly}(n)$ is the number of ancilla qubits used by the reversible computation. This is precisely the reflection that applies a -1 phase to eigenstates with energy at most E_j and acts as the identity otherwise. Operationally, this is done by reversibly computing the T-bit energy value $(H_j)_{ii}$ into an ancilla register, performing an inequality test to determine whether $(H_j)_{ii} \leq E_j$, applying a phase flip conditioned on the result of this test, and then uncomputing all ancilla registers. More details for this can be found in Appendix C.

Step 2: Fixed-point amplitude amplification. Assume first that exact reflection operators R_1 and R_2 are available. If the target overlap satisfies

$$\|\Pi_2 |\psi_1\rangle\| \geq \sqrt{p}, \tag{39}$$

then fixed-point amplitude amplification [13], using controlled versions of R_1 and R_2, constructs a unitary V such that

$$\left\| V\left(|\psi_1\rangle |0\rangle^{\otimes a}\right) - \frac{\Pi_2 |\psi_1\rangle}{\|\Pi_2 |\psi_1\rangle\|} |0\rangle^{\otimes a} \right\| \leq \frac{\delta}{2}. \tag{40}$$

The construction uses

$$L = O\left(\frac{\log(1/\delta)}{\sqrt{p}}\right) \tag{41}$$

calls to controlled-R_1 and the same number of calls to controlled-R_2.

In practice, the exact reflections R_1 and R_2 are replaced by approximate implementations R'_1 and R'_2. Let U denote the resulting unitary obtained by using R'_1 and R'_2 inside the fixed-point amplitude amplification sequence. Then, for an appropriate choice of the reflection precision δ', one obtains

$$\left\| U\left(|\psi_1\rangle |0\rangle^{\otimes a}\right) - \frac{\Pi_2 |\psi_1\rangle}{\|\Pi_2 |\psi_1\rangle\|} |0\rangle^{\otimes a} \right\| \leq \delta. \tag{42}$$

Therefore, the jump operator can be implemented successfully.

The discussion above shows that, given the required reflection operators and the overlap lower bound, the generic jump primitive can be implemented successfully. In the present algorithm, this primitive is applied twice: $-\frac{X}{n} \longrightarrow H_b^\eta$, and $H_b^\eta \longrightarrow \frac{H}{|E^*|}$. Each jump involves one classical Hamiltonian and the non-classical intermediate Hamiltonian H_b^η. Therefore, the only nontrivial block encoding required for the jump implementation is the block encoding of the intermediate Hamiltonian H_b^η. I now describe how to construct the block encoding of H_b^η up to exponentially small precision. For the implementation, I will stick to the method presented by Dalzell et al [2], even though they suggest further optimizations to the block encoding circuit can be made by using useful circuit primitives developed in Ref. [17].

Proposition 2 (Block encoding of H_b^η)**.** *Suppose that H is an efficiently computable cost function on n bits, with known optimal value E^*. Let $b < 1$, and let $g : [-1, \infty) \to [-1, 0]$ be an efficiently computable function. Then, for any $\delta \geq \exp(-\text{poly}(n))$, there exists a unitary U that δ-approximates, in operator norm, a $(1 + b, \text{poly}(n))$-block encoding of*

$$H_b = -\frac{X}{n} + b\, g\left(\frac{H}{|E^*|}\right). \tag{43}$$

The circuit complexity of U is polynomial in n. Moreover, controlled-U and controlled-controlled-U can also be implemented efficiently.

Implementation. The Hamiltonian in Eq. (43) consists of two terms. The first term, $-X/n$, is diagonal in the Hadamard basis. The second term, $g\left(\frac{H}{|E^*|}\right)$, is diagonal in the computational basis. Therefore, the two terms can be block encoded separately and then combined using a linear-combination-of-block-encodings construction.

Starting with the transformed cost term. Define $h(z) := g\left(\frac{H(z)}{|E^*|}\right)$. Assume that $h(z)$ can be represented to the required accuracy using $T = \mathrm{poly}(n)$ bits. Since $h(z) \in [-1, 0]$, then define $\theta_z \approx \arcsin(h(z))$, where θ_z is represented using $P = \mathrm{poly}(n)$ bits. Then, the block encoding is constructed by using reversible classical arithmetic in superposition. Starting from a computational-basis eigenstate $|z\rangle$ and ancilla registers initialized to zero, one performs arithmetic operations perform the mapping:

$$|0\rangle^{\otimes T} |0\rangle^{\otimes P} |0\rangle |0\rangle^{\otimes A} |z\rangle \longmapsto |0\rangle^{\otimes T} |0\rangle^{\otimes P} (\sin(\theta_z) |0\rangle + \cos(\theta_z) |1\rangle) |0\rangle^{\otimes A} |z\rangle . \tag{44}$$

Here $A = \mathrm{poly}(n)$ denotes the number of additional work qubits required for the reversible arithmetic. Projecting the rotation ancilla onto $|0\rangle$ gives amplitude $\sin(\theta_z)$, which approximates $h(z)$. Therefore, the unitary defined by Eqs. (44) gives a $(1, \mathrm{poly}(n))$-block encoding of $g\left(\frac{H}{|E^*|}\right)$, up to error $\exp(-\Omega(P))$ in operator norm.

The block encoding of the transverse-field term, $-\frac{X}{n}$, is constructed similarly. This operator is diagonal in the Hadamard basis. Therefore, the operator can be easily diagonalized in the computational basis. Its diagonal entries can then be computed efficiently, and the same reversible-arithmetic construction gives a $(1, \mathrm{poly}(n))$-block encoding of $-X/n$.

Finally, since

$$H_b = -\frac{X}{n} + b\, g\left(\frac{H}{|E^*|}\right), \tag{45}$$

the two block encodings can be combined using the linear-combination-of-block-encodings construction. The coefficients of the two terms are 1 and b, so the resulting unitary is a $(1 + b, \mathrm{poly}(n))$-block encoding of H_b. Since $b < 1$ and all arithmetic operations are efficient, the total circuit complexity remains polynomial in n. Controlled and controlled-controlled versions are obtained by controlling all operations in the block-encoding and linear-combination circuits.

4 Attempting a SWAP-less implementation of the super-Grover quantum algorithm

In the following I first break down the implementation unitaries that were introduced in Sec.3.4, isolating the main primitives that are required to implement the algorithm of Dalzell et al. [2]. I then try and compile these primitives assuming a LNN connectivity using temporal parity computing.

4.1 Summary of required operations

With the intent of implementing a SWAP-less version of the algorithm of Dalzell et al. [2] assuming LNN connectivity, it is useful to first isolate the circuit primitives that appear in the implementation. In the present case, the relevant primitives arise from the implementation of the two jump steps, where the details are fleshed out in Sec. 3.4. Once these primitives have been identified, they provide the natural starting point for applying the SWAP-less compilation framework.

I first consider the reflection associated with H_b^η. This reflection is implemented using the QSVT applied to a block encoding of the shifted and normalized Hamiltonian. Therefore, it requires controlled versions of both the block encoding of $\frac{-E_j-\Delta/2}{2\kappa_j}I$ and H_b^η, and the QSVT phase operators Π_ϕ. Since this reflection is itself used inside fixed-point amplitude amplification, the corresponding primitives must be implemented with one additional level of control. Thus, the fixed-point amplitude amplification circuit requires controlled-controlled block encodings of $\frac{-E_j-\Delta/2}{2\kappa_j}I$ and of H_b^η, as well as controlled versions of the QSVT phase operators Π_ϕ.

Secondly, I consider the reflections associated with the diagonal Hamiltonians H_{init} and H_{final}. Since these Hamiltonians are diagonal in the computational basis and their energy values are assumed to be available through lookup tables, their reflections can be implemented by computing the energy value, performing an inequality test, applying a phase flip conditioned on the result of this test, and then uncomputing the auxiliary registers. When these reflections are used inside fixed-point amplitude amplification, they need to be controlled.

Putting everything together I have that the fundamental primitives required to implement the super Grover algorithm are:

- Reflection operator for H_b^η;
  - Multi-controlled NOT gates, $C^{n+2}X$;
  - controlled-controlled sin and arcsin implemented as arithmetic operations;
  - controlled-controlled cascades of controlled R_y rotations;
- QSVT phase operator Π_ϕ
  - one-to-many CNOT layer;
  - multi-controlled NOT gates, C^mX;
  - controlled phase rotation $CP(2\phi)$;
- Reflection operator for H_{init} and H_{final};
  - Multi-controlled NOT gates, $C^{n+1}X$;
  - controlled inequality tests and its inverse;
  - controlled Pauli$Z$.

The full detail for the derivation of this list can be found in Appendix D.

4.2 Reflection operator for H_b^η

In this section, I focus on the reflection operator associated with the intermediate Hamiltonian H_b^η and isolate the main circuit primitives that must be compiled. In particular, I attempt to compile the C^nX operations, the controlled-controlled arcsin and sin arithmetic blocks, and the controlled-controlled cascade of rotations. A high-level circuit implementation is shown in Fig. 4.

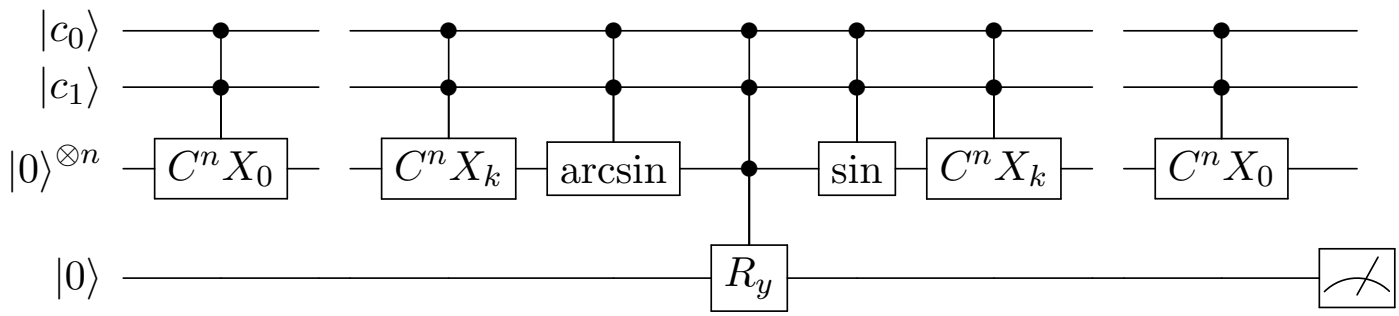


Figure 4: Schematic overview of circuit implementation of the controlled reflection operator associated with H_b^η. The first two wires are the control qubits, the third wire denotes the main system register, and the final wire is the rotation ancilla. The double-controlled R_y gate represents a double-controlled cascade of R_y rotations.

4.2.1 Multi-controlled NOT gates

The first element that we will tackle is probably the most important element present in the entire algorithm, the multi-controlled NOT gate in the LNN setting. A natural first approach is to express the multi-controlled NOT gate in terms of diagonal logical Z operations and then attempt to apply parity-label tracking. The construction is based on the phase-gadget decomposition of multi-controlled Z gates presented in Chapter 10 of [18]. Using $X = HZH$, one can first reduce the problem to the decomposition of a multi-controlled Z gate as

$$C^nX = H_tC^nZH_t, \tag{46}$$

where H_t acts on the target qubit. If $Q = \{c_1, \ldots, c_n, t\}$ denotes the set of all qubits involved in the gate, then C^nZ applies a phase -1 only to the computational basis state in which all qubits in Q are equal to 1. The projector onto this all-one state is $\prod_{q\in Q}(I - Z_q)/2$, and therefore, up to a global phase, the multi-controlled Z gate can be written as an exponential of products of Pauli-Z operators. Expanding the product gives

$$C^nZ \sim \prod_{\emptyset\neq S\subseteq Q} \exp\left(\frac{i\pi}{2^{n+1}}(-1)^{|S|}\prod_{q\in S} Z_q\right), \tag{47}$$

where $\sim$ denotes equality up to a global phase. Equivalently, C^nZ can be expressed as a product of logical Z-parity rotations over all non-empty subsets $S \subseteq Q$. Since there are $n+1$ qubits in Q, the number of non-empty subsets, and therefore the total number of logical Z operations, is $2^{n+1} - 1$.

At first sight, Eq. (47) appears well suited to parity-label tracking. Each factor is generated by a product of Pauli-Z operators, and can therefore be interpreted as a logical rotation associated with the parity of the qubits in S. However, this approach quickly encounters a limitation. The extended LHZ construction used in this work naturally tracks one and two-body parity labels, whereas the decomposition in Eq. (47) contains parity rotations of all orders.

One possible formal workaround is to reduce each many-body parity rotation to a single-body rotation by computing the corresponding parity into an ancilla. For a subset $S = \{q_1, \ldots, q_m\}$, consider the logical parity rotation $\bar{R}_{Z_S}(\theta) = \exp[-i\theta Z_{q_1}Z_{q_2}\cdots Z_{q_m}/2]$. Using a clean ancilla a, this can be implemented as

$$\bar{R}_{Z_S}(\theta) = \left[\prod_{j=1}^{m}\mathrm{CNOT}(q_j, a)\right]\bar{R}_{Z_a}(\theta)\left[\prod_{j=m}^{1}\mathrm{CNOT}(q_j, a)\right], \tag{48}$$

However, this observation does not by itself provide a useful extended-LHZ compilation strategy for a generic C^nX gate. The reason is that the required CNOTs are not naturally captured by the extended LHZ architecture. Thus, while the phase-gadget decomposition is algebraically systematic, the extended LHZ architecture in its present form does not appear to be the appropriate tool for

directly compiling the full generic multi-controlled NOT gate. A possible direction would be to consider an enlarged version of the LHZ construction that tracks higher-order parity labels. Such a generalized architecture could, in principle, make the higher-order terms appearing in Eq. (47) directly accessible. This, however, is beyond the scope of the present notes.

Instead, I focus on the smallest nontrivial and most useful case: the Toffoli gate. The Toffoli gate is the case of a doubly-controlled NOT, and its diagonal counterpart is the three-qubit controlled-controlled-Z gate. In this case, Eq. (47) contains only one three-body contribution. Therefore, temporal parity computing on the extended LHZ architecture can be used to efficiently compile the one and two body terms in the decomposition of the controlled-controlled-Z gate, whilst the three-body term can be implemented by a linear cascade of CNOT gates and a single-qubit Z rotation on an ancilla qubit, as in Eq. (48). The resulting circuit can be arranged efficiently on a LNN architecture, yielding the LNN Toffoli construction with 10 CNOTs, a CNOT depth of 10, 9 single qubit rotations and a single qubit depth of 3. The circuit is shown in Fig. 5a.

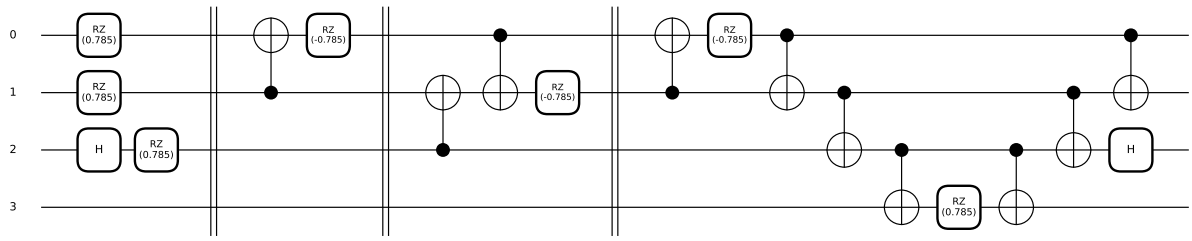
The code used to compile the Toffoli is organized into three components. The first part generates the logical gate, found in `Scripts/Toffoli_gates.py`, representation of the C^nX operation by using Eq. (46) and expanding the multi-controlled Z as a product of logical Z-parity rotations as in Eq. (47). This produces a gate list containing an initial Hadamard on the target, a sequence of logical R_Z rotations acting on supports of different sizes, and a final Hadamard. The three body logical R_Z rotation is decomposed into CNOTs on an ancilla and a local R_Z as shown in Eq. (48). The second part, found in `Scripts/Toffoli_spanning_lines.py`, constructs the spanning-line schedule. The spanning lines are generated for the system consisting of the control qubits and the target qubit, while the clean ancilla is appended to each spanning line as an additional singleton label. The schedule follows the left-to-right LHZ spanning-line construction used for the QFT, but is stopped once the first label of the final line reaches $(n-1, n)$, where n denotes the target qubit. A modified final update is then applied so that labels involving the target are kept fixed, giving a final spanning line adapted to the Toffoli compilation procedure. Finally, the compiler, found at `Scripts/Toffoli_compiler.py`, uses these spanning lines to apply all one and two-body R_Z rotations as soon as the corresponding parity labels become local. Once the spanning line have all been scanned, the code implements the remaining three-body term by simply applying a CNOT ladder from the first qubit to the ancilla, applying the local R_Z rotation before un-computing the CNOTs and applying a final hadamard.

This is sufficient for compiling larger multi-controlled NOT gates using the "sawtooth" construction of Ref. [19]. In that work, multi-controlled NOT gates, C^nX, are decomposed into a sequence of $2n$ Toffoli gates, with the addition of n ancilla qubits, arranged in a LNN pattern, as illustrated in Fig. 5b. Combined with the LNN Toffoli developed before, this produces a multi controlled NOT C^nX with: $20n$ CNOTs, a CNOT depth of $20n$, $18n$ single qubit rotations and a single qubit depth of $6n$ for a single C^nX operation. I note that when stacking multiple sawtooth construction together, optimizations can be done to further reduce the number of required Toffoli gates. Furthermore, I note that when applying the final Toffoli, the target qubit has to be previously swapped in the correct position to ensure the correct application using LNN connectivity. Therefore, although we do not obtain a direct extended-LHZ compilation of a generic C^nX gate from the full phase-gadget expansion, the ability to implement Toffoli gates with LNN connectivity is enough to implement the sawtooth multi-controlled NOT circuit. In this sense, the Toffoli construction provides an indirect but systematic route to implementing multi-controlled NOT gates on LNN hardware.

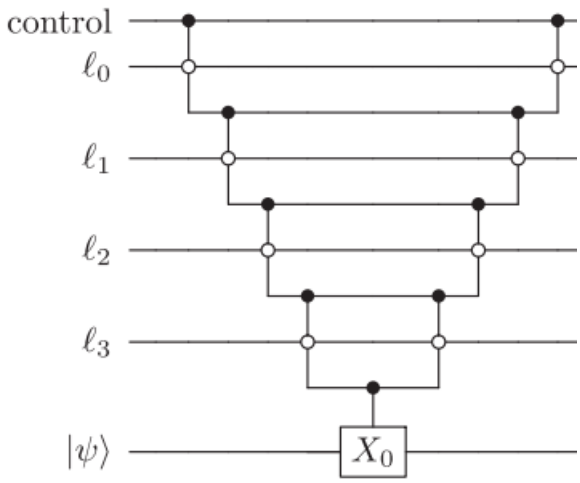
4.2.2 Controlled-controlled arcsin and sin

For the implementation of the controlled-controlled arcsin and sin we first consider the non-controlled version of the arithmetic operation. The controlled-controlled version can then be obtained by adding the required external controls to the arithmetic circuit, although this substantially increases the compilation difficulty on a LNN architecture.

For the implementation of the sin and arcsin blocks, we follow the approach of Ref. [20]. The main idea is to reduce the evaluation of elementary functions to the reversible evaluation of polynomial



(a)



(b)

Figure 5: Indirect LNN implementation route for multi-controlled NOT gates. (a) LNN implementation of the Toffoli gate with an ancilla qubit using temporal parity computing with the extended LHZ architecture. Code related to the development of this gate can be found at the following Github [3] in the `Notebooks/Toffoli.ipynb` notebook. (b) Sawtooth construction of Ref. [19], which decomposes a generic C^nX gate into nearest-neighbour Toffoli gates using ancilla qubits. Alongside the Toffoli implementation in LNN connectivity in Fig. 5a, allows to implement C^nX in LNN connectivity using the gate set H, R_Z, R_X and CNOT.

approximations. For example, one approximates

$$f(x) \approx P(x) = \sum_{i=0}^{d} a_i x^i,$$

where f is either sin or arcsin on the relevant interval. The polynomial can then be evaluated reversibly using Horner's scheme,

$$P(x) = a_0 + x\left(a_1 + x\left(a_2 + \cdots + x(a_{d-1} + xa_d)\right)\right).$$

This representation is useful because each step consists of a multiplication by x, followed by an addition of a coefficient a_i. Therefore, the arithmetic circuit reduces to repeated additions and multiplications.

The addition steps appearing in the polynomial evaluation can be implemented using a QFT-based adder [21]. In this approach, one applies the QFT to the arithmetic register, implements the addition of the required coefficient through a sequence of controlled phase rotations, and then applies the inverse QFT. Therefore, within the non-controlled implementation of the sin and arcsin arithmetic blocks, the addition subroutines can be implemented using the same LHZ construction developed in Sec. 2.3.

Multiplication can also be built from QFT-based arithmetic. Indeed, a reversible multiplier can be decomposed into a sequence of controlled additions [20]. If each addition is implemented using a QFT-based adder, this gives a QFT-based multiplication circuit. However, compared with addition, multiplication require controlled additions. As a result, while the addition subroutines are naturally compatible with temporal parity computing, the multiplication subroutines are less straightforward.

If we now go back to the reflection subroutine, we find the sin and arcsin arithmetic blocks must themselves be doubly controlled by external qubits. This means that the entire polynomial-evaluation circuit, including the QFT-based additions and multiplication, must be promoted to controlled-controlled operations. This makes the SWAP-less compilation impossible on a LNN architecture. Nevertheless, the LHZ implementation of the QFT-based adder remains advantageous even for all-to-all connected devices due to its lower total circuit depth than the standard QFT implementation. For this reason, even when the controlled version of the sin and arcsin arithmetic blocks is difficult to compile swaplessly under LNN constraints, the LHZ implementation of the underlying QFT-based addition subroutines remains a useful candidate primitive, and may still be preferable even on hardware with all-to-all connectivity.

4.2.3 Controlled-controlled cascade of R_y rotations

For the implementation of the double controlled cascade of R_y rotations, no obvious strategy was identified for exploiting temporal parity computing to realize the operation directly on a LNN architecture. The difficulty is that the cascade requires a sequence of rotations controlled by the angle register, and this structure does not naturally map onto the parity-tracking operations for which the SWAP-less compilation framework is most effective. As a result, the most straightforward implementation remains the standard approach based on a SWAP network, in which the relevant control qubits are brought next to the target rotation qubit as needed.

4.3 QSVT phase operator Π_ϕ

In this section, I focus on the QSVT phase operator Π_ϕ and isolate the main circuit primitives that must be compiled. In particular, I study the compilation of long-range CNOTs, while the implementation of the multi-controlled NOT gates C^mX on a LNN architecture has already been discussed in Sec. 4.2.1. I do not discuss the controlled phase rotation $CP(2\phi)$ in detail, since it is a single two-qubit gate once the relevant controls have been computed. A high-level circuit implementation is shown in Fig. 6.

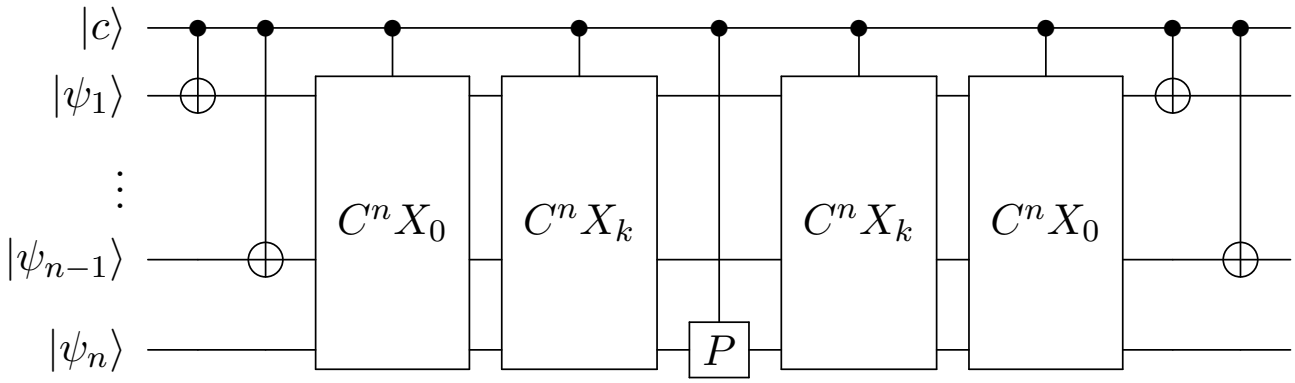


Figure 6: Schematic overview of the QSVT phase operator Π_ϕ. The first wire is the control qubits, the wires $|\psi_i\rangle$ represent the wires in the main register.

4.3.1 One-to-many CNOT layer

The QSVT phase operator Π_ϕ requires a one-to-many CNOT layer operations because the phase factor must be applied conditionally on the block-encoding ancillas being in the $|0\rangle^{\otimes m}$ state. Equivalently, this requires a multi-control on the zero state. A convenient way to implement such a zero-controlled operation is to apply X gates before and after the controlled phase operation, thereby converting

controls on $|0\rangle$ into controls on $|1\rangle$. However, because this construction appears inside the controlled reflection operator, these X gates become controlled operations themselves, and therefore appear as a one-to-many CNOT layer.

For this primitive, I do not see an obvious way to exploit temporal parity computing to obtain a SWAP-less implementation. The operation consists of applying controlled bit flips between spatially separated qubits, rather than evolving or tracking a useful parity label through the architecture. As a result, the most direct implementation remains a standard SWAP-based strategy, in which the relevant qubits are moved along the LNN chain until the required CNOTs can be applied locally. This makes the one-to-many CNOT layer one of the primitives for which the SWAP-less compilation method does not appear to provide an immediate advantage.

4.4 Reflection operator for H_{init} and H_{final}

In this section, I focus on the exact reflection operators associated with the diagonal Hamiltonians H_{init} and H_{final}. The main circuit primitives appearing in this construction are the multi-controlled NOT gates $C^{n+1}X$, the controlled inequality test and its inverse, and the controlled Pauli-Z operation. The implementation of multi-controlled NOT gates on a LNN architecture has already been discussed in Sec. 4.2.1, while the controlled Pauli-Z is a single two-qubit gate once the comparison bit has been computed. Therefore, the only remaining primitive that requires further analysis is the controlled inequality test.

4.4.1 Controlled inequality tests and its inverse

As in the discussion of the sin and arcsin arithmetic blocks in Sec. 4.2.2, we first consider the non-controlled version of the operation. Given two T-bit fixed-point numbers a and b, the goal of the inequality test is to compute a comparison bit indicating whether $a \leq b$. This can be implemented reversibly by reducing the comparison to a subtraction: one computes $a - b$ into an auxiliary register and extracts the sign bit, or equivalently the borrow bit, of the subtraction. In the present reflection circuit, the two quantities being compared are the computed energy value $(H_j)_{ii}$ and the threshold E_j.

The subtraction can be implemented using a QFT-based adder/subtractor [21], and therefore can be compiled using the QFT construction built in Sec. 2.3. The main complication arises from the fact that, in the full reflection circuit, the inequality test must itself be controlled by an external ancilla. The same considerations discussed in Sec. 4.2.2 therefore apply: the SWAP-less compilation is impossible on a LNN architecture, but nevertheless, the LHZ implementation of the QFT-based adder remains advantageous even for all-to-all connected devices due to its lower total circuit depth than the standard QFT implementation.

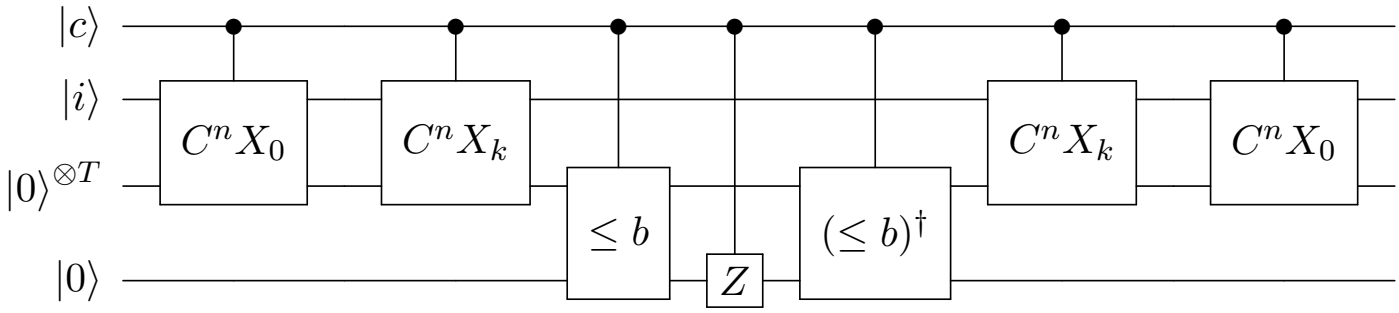


Figure 7: Schematic circuit for the exact reflection operator associated with the diagonal Hamiltonians H_{init} and H_{final}. The first wires is the control qubit, the second wire is the index register, whilst the third wire is the T wire register used for the arithmetic operations and the final qubit stores the solution to the inequality test.

5 Conclusions

In these notes I investigated whether the temporal-parity compilation framework of Ref. [1] can be used to implement the super-Grover optimization algorithm of Ref. [2] on a LNN architecture without SWAP gates. As a validation step, I implemented the extended-LHZ spanning-line construction and reproduced the SWAP-less compilation of the QFT and of the QAOA problem and mixer unitaries, confirming the expected resource scalings numerically.

I then analyzed the primitives required by the jump implementation of Ref. [2]. From an algorithmic perspective, the central component is the jump operator between two Hamiltonians H_1 and H_2, which prepares the normalized projection of an eigenstate of H_1 onto the relevant low-energy eigenspace of H_2. When this construction is applied to the Hamiltonians considered in the optimization algorithm, the implementation reduces to a collection of circuit subroutines involving controlled NOT operations, controlled arithmetic operations, controlled rotations, reflection operators.

The compilation analysis showed that some of these primitives can be implemented efficiently within the SWAP-less framework, while others are much less naturally compatible with temporal parity computing. In particular, I was able to derive a SWAP-less implementation of a multi-controlled NOT gate C^nX on LNN connectivity using $20n$ CNOT gates, CNOT depth $20n$, $18n$ single-qubit rotations, and single-qubit depth $6n$ for a single C^nX operation. I also identified useful subroutines appearing in the arithmetic circuits, such as the inequality test, that employ the QFT and can therefore be implemented without SWAP gates using the LHZ-based QFT construction. Nevertheless, the overall implementation of the full optimization algorithm on a LNN architecture without SWAP gates remains difficult, mainly because the QSVT and block-encoding routines introduce controlled arithmetic, controlled rotations, and high-order controlled operations that are not native to the temporal-parity framework.

Several observations emerged from this analysis. First, an extension of the extended LHZ code to general n-qubit parities, together with systematic parity-label, would likely allow for the implementation of operations such as $C^{n+1}X$. Second, the compilation of QSVT-based, block-encoding algorithms using temporal parity computing appears to be somewhat unnatural. The purpose of block encoding is to enlarge the working register so that one can represent operations such as sums of matrices, rather than only products, and implement matrix functions as polynomial transformations built from sums of polynomial terms. This is quite different from the setting in which temporal parity computing is most naturally effective, namely, as emphasized by Klaver et al. [1], the implementation of algorithms requiring the interactions specific to transverse-field Ising Hamiltonians. Given these considerations, a more natural future direction may be to apply the two-jump idea of Dalzell et al. [2] through a conventional adiabatic evolution implemented through Trotterization. This direction is particularly natural for Ising spin-glass optimization, which Dalzell et al. [2] identify as one of the problem families where their strategy can be proved to work well due to favorable spectral properties, and where the resulting algorithm appears to offer an advantage over the best-known classical approaches. In fact, the QAOA compilation analysis already provides the main subroutines required to implement such an adiabatic protocol for an Ising model. In that setting, the bounds derived in Dalzell et al. [2] could help determine the appropriate size of the jumps, while the resulting time-evolution circuits would be a much better match for SWAP-less compilation through temporal parity computing.

A Brief overview of the quantum singular value transformation

Quantum singular value transformation (QSVT) is a general framework for applying polynomial transformations to the singular values of a block-encoded matrix [12]. It is one of the main algorithmic primitives used in modern quantum algorithms, because many tasks can be reduced to transforming the spectrum of a matrix: Hamiltonian simulation, eigenvalue filtering, matrix inversion, and amplitude amplification can all be viewed through this lens. In this appendix, I give a brief overview of the idea. The goal is to explain what the algorithm is designed to do and how it is implemented at the circuit level. The material is based on the following set of lecture notes [22].

A.1 Block encodings

The starting point of QSVT is the notion of a block encoding. A block encoding is a way of embedding a generally non-unitary matrix into a larger unitary operation acting on additional ancilla qubits. Let $A \in \mathbb{C}^{N\times N}$ be a matrix. A unitary U_A is said to be a (α, m)-block encoding of A if

$$\left(\langle 0|^{\otimes m} \otimes I\right) U_A \left(|0\rangle^{\otimes m} \otimes I\right) = \frac{A}{\alpha}, \tag{49}$$

where $\alpha \geq \|A\|$ is a normalization factor and m is the number of ancilla qubits. Equivalently, with respect to the ancilla state $|0\rangle^{\otimes m}$, the matrix A/α appears as the top-left block of the larger unitary U_A as shown below:

$$U_A = \begin{pmatrix} A/\alpha & \cdot \\ \cdot & \cdot \end{pmatrix}. \tag{50}$$

A.2 Singular value transformations

For a general matrix $A \in \mathbb{C}^{N\times N}$, the relevant spectral decomposition is the singular value decomposition

$$A = W\Sigma V^\dagger. \tag{51}$$

Here W and V are unitary matrices, and

$$\Sigma = \mathrm{diag}(\sigma_0, \sigma_1, \ldots, \sigma_{N-1}) \tag{52}$$

is a diagonal matrix containing the singular values of A, with $\sigma_i \geq 0$. The columns of V are the right singular vectors of A, while the columns of W are the left singular vectors. Equivalently, if $|v_i\rangle$ and $|w_i\rangle$ denote the corresponding right and left singular vectors, then

$$A\,|v_i\rangle = \sigma_i\,|w_i\rangle\,, \qquad A^\dagger\,|w_i\rangle = \sigma_i\,|v_i\rangle\,. \tag{53}$$

Given a scalar function $f : \mathbb{R}_+ \to \mathbb{C}$, one can define a singular value transformation of A by applying f directly to the singular values:

$$f(\Sigma) = \mathrm{diag}\big(f(\sigma_0), f(\sigma_1), \ldots, f(\sigma_{N-1})\big). \tag{54}$$

The corresponding transformed matrix is

$$f^{\mathrm{SV}}(A) = W f(\Sigma) V^\dagger. \tag{55}$$

A.3 Quantum singular value transformation

Given an (α, m)-block encoding U_A of a matrix A, the aim of QSVT is to construct a new unitary whose projected block is a polynomial transformation of the singular values of the encoded matrix:

$$p^{\mathrm{SV}}\left(\frac{A}{\alpha}\right). \tag{56}$$

Here p is a polynomial chosen to approximate the desired scalar function on the singular values of A/α.

At a high level, QSVT is implemented by alternating applications of the block encoding U_A, or its adjoint $U_A^\dagger$, with phase rotations associated with the block-encoding subspace. A degree-d QSVT sequence contains d calls to U_A or $U_A^\dagger$, interleaved with $d+1$ phase rotations. The sequence is parameterized by a list of phases

$$\Phi = (\phi_0, \phi_1, \ldots, \phi_d), \tag{57}$$

where d is the degree of the polynomial transformation. The phases Φ are determined by a phase-synthesis procedure. The usual workflow is first to approximate the desired scalar function p^{SV} by an admissible polynomial $p(x)$, expressed in a Chebyshev basis,

$$p(x) = \sum_{k=0}^{d} c_k T_k(x).$$

where $T_k(x)$ is the kth order Chebyshev polynomial of the first kind. The QSVT phases are then obtained by solving the inverse problem of finding a phase sequence Φ whose associated QSVT polynomial is $p(x)$. This can be done using specialized QSVT phase-finding algorithms based on polynomial factorization or numerical optimization.

The phase rotations are usually written as

$$\Pi_\phi = e^{i\phi(2\Pi - I)}, \tag{58}$$

where

$$\Pi = |0\rangle^{\otimes m} \langle 0|^{\otimes m} \otimes I. \tag{59}$$

Here Π is the projector onto the block-encoding subspace. Equivalently, Π_ϕ acts as a controlled phase operation associated with this subspace: it applies a phase $e^{i\phi}$ on the subspace where the block-encoding ancillas are in the state $|0\rangle^{\otimes m}$ and a phase $e^{-i\phi}$ on the orthogonal subspace. In the special case where the block-encoding projector is associated with a single ancilla qubit, these operators reduce, up to a global phase, to single-qubit Z-rotations on that ancilla.

A schematic QSVT sequence, suppressing convention-dependent distinctions between input and output projectors, has the form

$$U_\Phi = \Pi_{\phi_0} \left(U_A \Pi_{\phi_1} U_A^\dagger \Pi_{\phi_2} U_A \Pi_{\phi_3} \cdots \right) \Pi_{\phi_d}, \tag{60}$$

up to ordering, parity, and phase-convention choices. Whether the sequence begins or ends with U_A or $U_A^\dagger$, together with the number of alternating applications, determines the parity of the polynomial that can be implemented. A generic high-level circuit structure is shown below:

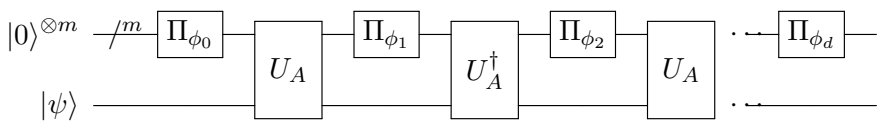


Figure 8: Schematic high-level structure of a QSVT circuit. The block encoding U_A and its adjoint $U_A^\dagger$ are interleaved with phase rotations Π_{ϕ_j} associated with the block-encoding subspace. The phases $\Phi = (\phi_0, \ldots, \phi_d)$ are chosen so that the resulting unitary block-encodes a polynomial singular-value transformation of A/α.

Not every polynomial can be implemented by QSVT. The target polynomial must satisfy certain admissibility conditions. At the level of this overview, the most important requirements in the standard real-polynomial formulation are that p has degree at most d, has parity matching the sequence length,

$$p(-x) = (-1)^d p(x), \tag{61}$$

and is bounded in magnitude on the interval $[-1, 1]$,

$$|p(x)| \leq 1. \tag{62}$$

More precise QSVT theorems include additional technical conditions ensuring that p can be completed to a unitary transformation.

The output of the QSVT circuit is a new unitary U_Φ whose block-encoded action is

$$\left(\langle 0|^{\otimes m} \otimes I\right) U_\Phi \left(|0\rangle^{\otimes m} \otimes I\right) = p^{\mathrm{SV}}\left(\frac{A}{\alpha}\right), \tag{63}$$

up to implementation error. Thus, QSVT converts a block encoding of A/α into a block encoding of a polynomial transformation of its singular values. When p is chosen to approximate a desired function f, this gives an approximation to $f^{\mathrm{SV}}(A/\alpha)$.

B Construction of reflection operators

Here I will give further details about how the authors construct the reflection operator. First define the step function $q(x)$:

$$q(x) = \begin{cases} -1, & x \leq 0, \\ +1, & x \geq 0. \end{cases} \tag{64}$$

Let R_1 be the reflection that applies a -1 phase to the eigenstate $|\psi_1\rangle$ of H_1 and acts as the identity on the orthogonal subspace. Similarly, let R_2 be the reflection that applies a -1 phase to the eigenstates of Π_2 and acts as the identity on the orthogonal subspace.

Assuming that the relevant low-energy subspace of H_j is separated from the rest of the spectrum by the energy promise Δ_j, these reflections can be written as functions of the Hamiltonians:

$$R_j = q\left(\frac{H_j - E_j - \Delta_j/2}{2\kappa_j}\right), \qquad j \in \{1, 2\}. \tag{65}$$

The shift by $E_j + \Delta_j/2$ places the midpoint of the promised spectral gap at zero, while the normalization by $2\kappa_j$ normalizes the shifted Hamiltonian so that its spectrum lies within the interval $[-1, 1]$

The exact step function in Eq. (33) is discontinuous, so it is replaced by a polynomial approximation. Let L denote the number of fixed-point amplitude-amplification steps to be used later, and define

$$\delta' = \frac{\delta}{4L}. \tag{66}$$

Then there exists an odd polynomial $q'_j : [-1, 1] \to [-1, 1]$ of degree

$$d = O\left(\log\left(1/\delta'\right) \frac{\kappa_j}{\Delta_j}\right) \tag{67}$$

such that

$$q'_j(x) \leq -1 + \delta' \qquad \text{for} \qquad x \leq -\frac{\Delta_j}{4\kappa_j}, \tag{68}$$

and

$$q'_j(x) \geq 1 - \delta' \qquad \text{for} \qquad x \geq \frac{\Delta_j}{4\kappa_j}. \tag{69}$$

The corresponding approximate reflection is therefore

$$R'_j = q'_j\left(\frac{H_j - E_j - \Delta_j/2}{2\kappa_j}\right). \tag{70}$$

Because q'_j is odd and bounded on $[-1, 1]$, it can be implemented using the QSVT.

C Exact reflections for classically diagonal Hamiltonians

Here I explain the steps that allows to perform the reflection operator R_j for a classical Hamiltonian H_j. Here classical means H_j is diagonal in either the computational basis or the Hadamard basis and its diagonal entries can be computed efficiently, the reflection can be implemented exactly using reversible classical arithmetic. This is the case covered by condition (ii).

Suppose first that H_j is diagonal in the computational basis. Then write its diagonal entries as

$$H_j |i\rangle = (H_j)_{ii} |i\rangle . \tag{71}$$

Assume that T bits are sufficient to represent each diagonal entry $(H_j)_{ii}$ to the desired precision, and that these entries can be computed in classical time poly(n). Then there exists a reversible circuit using $A = \text{poly}(n)$ ancilla qubits that computes the energy value in superposition:

$$|i\rangle |0\rangle^{\otimes A} \longmapsto |i\rangle |(H_j)_{ii}\rangle |0\rangle^{\otimes (A-T)} . \tag{72}$$

Here $|(H_j)_{ii}\rangle$ denotes a T-bit binary representation of the diagonal entry.

The next step is to compare this computed energy with the threshold E_j. This comparison is also performed reversibly. Introducing one additional ancilla bit, implement

$$|i\rangle |(H_j)_{ii}\rangle |0\rangle^{\otimes (A-T)} \longmapsto |i\rangle |(H_j)_{ii}\rangle |(H_j)_{ii} \leq E_j\rangle |0\rangle^{\otimes (A-T-1)} . \tag{73}$$

The bit $|(H_j)_{ii} \leq E_j\rangle$ stores the result of the inequality test. It is equal to $|1\rangle$ if the eigenvalue is at most E_j, and equal to $|0\rangle$ otherwise.

One then applies a Pauli-Z gate to this comparison bit. This introduces a phase -1 precisely when $(H_j)_{ii} \leq E_j$:

$$|(H_j)_{ii} \leq E_j\rangle \longmapsto (-1)^{(H_j)_{ii} \leq E_j} |(H_j)_{ii} \leq E_j\rangle . \tag{74}$$

Finally, the inequality test and energy-computation circuits are uncomputed. All ancilla registers are returned to the state $|0\rangle$, leaving the net transformation

$$|i\rangle |0\rangle^{\otimes A} \longmapsto (-1)^{(H_j)_{ii} \leq E_j} |i\rangle |0\rangle^{\otimes A} . \tag{75}$$

This is exactly the desired reflection operator

$$R_j = I - 2\Pi_{\leq E_j}, \tag{76}$$

where $\Pi_{\leq E_j}$ is the projector onto the eigenspace of H_j with energy at most E_j. If H_j is diagonal in the Hadamard basis rather than the computational basis, the same procedure is used after a change of basis.

D Derivation of fundamental primitives

I first consider the reflection associated with H_b^η. As mentioned in Sec. 4.1, the fixed point amplitude amplification of the reflection requires the controlled-controlled block encodings of $\frac{-E_j - \Delta/2}{2\kappa_j} I$ and of H_b^η, as well as controlled versions of the QSVT phase operators Π_ϕ.

The block encoding of H_b^η requires the reversible computation of the energy values, the arcsin operation, the application of a cascade of controlled R_y rotations, and the inverse arithmetic steps needed to uncompute the auxiliary registers.

For the diagonal Hamiltonians considered here, I assume that the energy values are provided explicitly through a classical lookup table, meaning that they can be implemented through a series of n multi-controlled NOT operation, which I denote as $C^n X$. In this representation, the energy-computation operation is its own inverse.

Consequently, the controlled-controlled block encoding of H_b^η requires the following primitives:

- $C^{n+2}X$;

- controlled-controlled computation of the rotation angles, computed as arcsin computation;
- controlled-controlled cascade of controlled R_y rotations;
- controlled-controlled inverse computation of the rotation angles, i.e. sin;

The block encoding of the scalar term

$$\frac{-E_j - \Delta/2}{2\kappa_j} I \tag{77}$$

is diagonal in the computational basis. Therefore, its implementation uses the same lookup-table and controlled-rotation structure as the diagonal terms appearing in the block encoding of H_b^η. Hence, in the fixed-point amplitude amplification circuit, the corresponding controlled-controlled block encoding requires the same types of primitives as the block encoding of H_b^η.

Next consider the phase operators appearing inside the QSVT sequence. For an (α, m)-block encoding, the QSVT phase operator is

$$\Pi_\phi = e^{i\phi(2\Pi - I)}, \qquad \Pi = |0\rangle^{\otimes m} \langle 0|^{\otimes m} \otimes I. \tag{78}$$

This operation applies a phase $e^{i\phi}$ to the block-encoding subspace and a phase $e^{-i\phi}$ to its orthogonal complement. Up to a global phase, it is equivalent to applying a phase $e^{i2\phi}$ only when the block-encoding ancilla register is in the state $|0\rangle^{\otimes m}$. Thus, it can be implemented as a multi-controlled phase gate controlled on $m-1$ block-encoding ancilla qubits and targeted at the remaining ancilla qubit.

$$C^{m-1}X, \qquad P(2\phi), \qquad C^{m-1}X, \tag{79}$$

where $P(2\phi) = \mathrm{diag}(1, e^{i2\phi})$. The controls are taken on the all-zero block-encoding ancilla state, which can be implemented using X gates before and after the controlled operation. When the QSVT phase operator is itself controlled by the outer fixed-point amplitude amplification circuit, one requires the controlled version of this multi-controlled phase operation, as well as the controlled version of the X gates before and after the controlled operation, which are simply CNOT gates. Therefore, in the fixed-point amplitude amplification circuit one requires: CNOT gates to set the control on 0, C^mX, a controlled phase gate, $CP(2\phi)$, C^mX and finally another round of CNOT gates to set the control on 0. Note that the CNOTs after the first and before the second C^mX cancel each other out. It is worth noting that these CNOTs are difficult to implement in the LNN architecture due to the fact they require long range interactions.

Finally, consider the reflections associated with the diagonal Hamiltonians H_{init} and H_{final}. Since these Hamiltonians are diagonal in either the computational or hadamard basis and their energy values are assumed to be available through lookup tables, their reflections can be implemented by computing the energy value, performing an inequality test, applying a phase flip conditioned on the result of this test, and then uncomputing the auxiliary registers. When these reflections are used inside fixed-point amplitude amplification, the following controlled primitives are required:

- Multi-controlled NOT gates, $C^{n+1}X$;;
- controlled inequality test and the controlled inverse;
- controlled Pauli-Z;

In summary, the complete implementation requires the following controlled primitives:

- Reflection operator for H_b^η;
  - Multi-controlled NOT gates, $C^{n+2}X$;
  - controlled-controlled sin and arcsin implemented as arithmetic operations;
  - controlled-controlled cascades of controlled R_y rotations;
- QSVT phase operator Π_ϕ;

  - one-to-many CNOT layer;
  - multi-controlled NOT gates, $C^m X$;
  - controlled phase rotation $CP(2\phi)$;

- Reflection operator for H_{init} and H_{final}
  - controlled inequality tests and its inverse;
  - controlled Pauli$Z$.

References

[1] B. Klaver et al., "Swap-less implementation of quantum algorithms," *Phys. Rev. A*, vol. 113, p. 012 443, 1 Jan. 2026.

[2] A. M. Dalzell, N. Pancotti, E. T. Campbell, and F. G. Brandão, "Mind the gap: Achieving a super-grover quantum speedup by jumping to the end," in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, ser. STOC '23, ACM, 2023, pp. 1131–1144.

[3] F. Ghisoni, *Swap-less compiler*, Available at `https://github.com/fghisoni/SWAPlessCompiler`, 2026.

[4] W. Lechner, P. Hauke, and P. Zoller, "A quantum annealing architecture with all-to-all connectivity from local interactions," *Science Advances*, vol. 1, no. 9, e1500838, Jul. 2026.

[5] M. Fellner, A. Messinger, K. Ender, and W. Lechner, "Universal parity quantum computing," *Phys. Rev. Lett.*, vol. 129, p. 180 503, 18 Oct. 2022.

[6] V. Bergholm et al., *Pennylane: Automatic differentiation of hybrid quantum-classical computations*, 2022.

[7] E. Farhi, J. Goldstone, and S. Gutmann, *A quantum approximate optimization algorithm*, 2014.

[8] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, *Quantum computation by adiabatic evolution*, 2000.

[9] T. Albash and D. A. Lidar, "Adiabatic quantum computation," *Rev. Mod. Phys.*, vol. 90, p. 015 002, 1 Jan. 2018.

[10] M. Born and V. Fock, "Beweis des adiabatensatzes," *Z. Phys.*, vol. 51, no. 3-4, pp. 165–180, 1928.

[11] M. B. Hastings, "A Short Path Quantum Algorithm for Exact Optimization," *Quantum*, vol. 2, p. 78, Jul. 2018.

[12] A. Gilyén, Y. Su, G. H. Low, and N. Wiebe, "Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics," in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2019, Phoenix, AZ, USA: Association for Computing Machinery, 2019, pp. 193–204.

[13] T. J. Yoder, G. H. Low, and I. L. Chuang, "Fixed-point quantum search with an optimal number of queries," *Phys. Rev. Lett.*, vol. 113, p. 210 501, 21 Nov. 2014.

[14] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96, Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219.

[15] A. Montanaro, "Quantum speedup of branch-and-bound algorithms," *Physical Review Research*, vol. 2, no. 1, Jan. 2020.

[16] T. D. Hansen, H. Kaplan, O. Zamir, and U. Zwick, "Faster k-sat algorithms using biased-ppsz," in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2019, Phoenix, AZ, USA: Association for Computing Machinery, 2019, pp. 578–589.

[17] Y. R. Sanders et al., "Compilation of fault-tolerant quantum heuristics for combinatorial optimization," *PRX Quantum*, vol. 1, p. 020 312, 2 Nov. 2020.

[18] A. Kissinger and J. van de Wetering, *Picturing Quantum Software: An Introduction to the ZX-Calculus and Quantum Compilation*. Cambridge University Press, 2024.

[19] R. Babbush et al., "Encoding electronic spectra in quantum circuits with linear t complexity," *Phys. Rev. X*, vol. 8, p. 041 015, 4 Oct. 2018.

[20] T. Häner, M. Roetteler, and K. M. Svore, "Optimizing quantum circuits for arithmetic," *arXiv preprint arXiv:1805.12445*, 2018.

[21] T. G. Draper, *Addition on a quantum computer*, 2000.

[22] N. W. Lin Lin, *Lecture notes on quantum algorithms for scientific computation*, Available at `https://math.berkeley.edu/~linlin/qasc/live_notes_0429.pdf`, 2026.